

JUST Undergraduate Final Admission System

A Project Report Submitted

In partial fulfillment of the requirements for the degree of
B.Sc. Engineering in Computer Science and Engineering



Submitted by

Nafis Ahamed

Student ID: 200129

Session: 2020–21

Supervised by

Dr. Mohammad Nowsin Amin Sheikh

Professor

Department of Computer Science and Engineering

Faculty of Engineering and Technology

Jashore University of Science and Technology

Jashore-7408, Bangladesh

April 2026

ACKNOWLEDGEMENT

First of all, I would like to express my deepest gratitude to the Almighty and Most Merciful God, by whose grace and blessings I have been able to complete this work successfully.

I would like to express my sincere appreciation and profound respect to my project supervisor, **Dr. Mohammad Nowsin Amin Sheikh**, Professor, Department of Computer Science and Engineering, Jashore University of Science and Technology (JUST), for his constant guidance, invaluable suggestions, and immense support throughout the duration of this project.

I am also truly grateful to the Dean, Faculty of Engineering and Technology, and the Chairman of our department, **Md. Kamrul Islam**, for their invaluable advice and constant encouragement in every aspect from the beginning to the end of this research work.

Finally, I owe my deepest thanks and gratitude to the pillars of my success—my beloved parents—for their endless blessings, sacrifices, and financial support, which have always motivated me to move forward.

Nafis Ahamed

Student ID: 200129

Preface

On the eve of the beginning of civilization, human beings had been striving tirelessly to discover, explore, and innovate unknown aspects of the world. This inherent curiosity and desire for advancement have driven humanity toward continuous progress and development. Over time, human beings have not only acquired theoretical knowledge but have also emphasized the importance of practical implementation, which plays a crucial role in solving real-world problems and improving the quality of life.

In the modern era, the field of Computer Science and Engineering has emerged as one of the most dynamic and influential disciplines, contributing significantly to technological advancement and digital transformation. The philosophy of the Computer Science and Engineering department is centered on equipping students with both theoretical understanding and practical skills. To achieve this goal, the department has introduced thesis and project programs that encourage students to engage in real-life problem-solving and innovative system development.

As a part of fulfilling the academic requirements of the undergraduate program, I have undertaken a project titled **“JUST Undergraduate Final Admission System”**. This project aims to develop a comprehensive web-based platform that simplifies and streamlines the admission process for undergraduate students. It focuses on efficiency, transparency, and user-friendliness, ensuring a better experience for both applicants and administrators.

Throughout the development of this project, I have conducted extensive research, analysis, and observation to understand the existing admission processes and identify their limitations. This has enabled me to design and implement a system that addresses real-world challenges effectively. The project has provided me with valuable opportunities to enhance my technical skills, problem-solving abilities, and understanding of software development methodologies.

I believe that this project will not only serve as an academic accomplishment but also contribute positively to the automation and digitalization of admission systems. The knowledge and experience gained during this work will undoubtedly play a vital role in my future career and professional development.

Contents

1	Introduction	1
1.1	Background	1
1.2	Motivation	2
1.2.1	Existing System	2
1.2.2	Limitations of the System	2
1.3	Objectives	3
2	Software Requirement Specification	4
2.1	Introduction	4
2.1.1	Purpose	4
2.1.2	Document Conventions	5
2.1.3	Intended Audience and Reading Suggestions	5
2.2	Overall Description	6
2.2.1	Product Perspective	6
2.2.2	Product Functions	6
2.3	User Classes and Characteristics	7
2.4	Constraints	8
2.5	External Interface Requirements	8
2.5.1	User Interfaces	8
2.5.2	Integration with Existing Data	9
2.5.3	Integration with Internal Modules	9
2.5.4	Authentication and Authorization	9
2.6	System Features: Admin Management	9
2.6.1	Description and Priority	9
2.6.2	Functional Requirements	9
2.7	System Features: Dean Office	10
2.7.1	Description	10
2.7.2	Functional Requirements	10
2.8	System Features: Hall Office	10

2.8.1	Description	10
2.8.2	Functional Requirements	11
2.9	System Features: Medical Office	11
2.9.1	Description	11
2.9.2	Functional Requirements	11
2.10	System Features: Registrar Office	11
2.10.1	Description	11
2.10.2	Functional Requirements	11
2.11	System Features: Applicant	12
2.11.1	Description	12
2.11.2	Functional Requirements	12
2.12	Non-Functional Requirements	12
2.12.1	Performance	12
2.12.2	Security	12
2.12.3	Scalability	12
2.12.4	Maintainability	12
2.12.5	Usability	12
2.12.6	Reliability	13
3	System Design and Methodology	14
3.1	Use Case Diagram	14
3.1.1	User Management	14
3.1.1.1	Sign in	15
3.1.1.2	View Applicant List	15
3.1.1.3	Update User Information	16
3.1.1.4	Delete User Information	18
3.1.1.5	View User Information	18
3.1.1.6	Fill-up Application Form	19
3.1.1.7	Download Application Form	20
3.1.2	Dean Office Management	22
3.1.2.1	Search Applicant	22
3.1.2.2	Show Applicant Profile	23
3.1.2.3	Verify Application Form	25
3.1.2.4	Verify Required Documents	25
3.1.2.5	Admit Applicant (Dean Clearance)	27
3.1.2.6	Reject Applicant	27

3.1.2.7	View Applicant Lists (All, Waited, Admitted) . . .	28
3.1.2.8	Pending Documents and Reminders	28
3.1.2.9	Seats Status and Data Export	29

List of Tables

3.1	Use Case Table: Sign in (UC-1.1)	15
3.2	Use Case Table: View Applicant List (UC-1.2)	16
3.3	Use Case Table: Update User Information (UC-1.3)	17
3.4	Use Case Table: Delete User Information (UC-1.4)	18
3.5	Use Case Table: View User Information (UC-1.5)	19
3.6	Use Case Table: Fill-up Application Form (UC-1.6)	20
3.7	Use Case Table: Download Application Form (UC-1.7)	21
3.8	Use Case Table: Search Applicant (UC-2.1)	23
3.9	Use Case Table: Show Applicant Profile (UC-2.2)	24
3.10	Use Case Table: Verify Application Form (UC-2.3)	25
3.11	Use Case Table: Verify Required Documents (UC-2.4)	26
3.12	Use Case Table: Admit Applicant (UC-2.5)	27
3.13	Use Case Table: Reject Applicant (UC-2.6)	28
3.14	Use Case Table: Applicant List Management (UC-2.7 - 2.9)	28
3.15	Use Case Table: Pending Documents Management (UC-2.11 - 2.13)	29
3.16	Use Case Table: Monitoring and Export (UC-2.14 - 2.15)	29

Chapter 1

Introduction

1.1 Background

Admission of a university is an indispensable activity in every educational activity and is as old as education itself. With more students applying for admission than in previous years, admission process for admissions officers has become increasingly time-consuming. More applications resulted in heavier paperwork and processing challenges. Every admission is routed through various faculty for evaluation, and the manual admission process caused difficulty in admission, and archiving.

In order to speed up and simplify the admission process, the development of the Automated Final Admission System is to enact document redesign and an automated management program. The proposed “JUST undergraduate final admission system” would store, route, and retrieve prospective and current application documents. The development of science and technology has immensely contributed to the growth of computers and the internet which has adversely increased the use and need for an automated applicant’s admission system.

Universities, Colleges, Companies, organizations, etc. are seeking better ways to distribute and disseminate information throughout the world. In order to achieve this, they develop and rely on online applications, which help them input, update, process, and disseminate information. The internet has increased the use of online applications because data could be easily and readily accessed as well as documented. This project is targeted at allowing admission of applicants’ decisions to be made faster and more efficiently. It looks at improving the present traditional system of the manual process (paper-based) which is manually documented. This type of project is a hot topic in the whole world to digitalize our education system.

1.2 Motivation

1.2.1 Existing System

There are several automated systems used for managing final admission processes in educational institutions. These systems differ in features, capabilities, and implementation depending on institutional needs and available resources.

Currently, JUST provides a web-based application titled “JUST Admission 2025”. While this system includes some useful functionalities, it is not a fully integrated or complete solution. Several important features are missing, such as admission confirmation, sending admission confirmation messages, and verification of required documents.

Another issue in the current system is that any user can register regardless of whether they have been selected for admission or not. This creates inconsistency and reduces system reliability. Additionally, the system lacks proper validation mechanisms, which makes it difficult to ensure the authenticity and correctness of submitted data.

1.2.2 Limitations of the System

The current system suffers from several limitations that affect its efficiency and usability.

First, there is no proper validation process implemented. As a result, users can input incorrect or incomplete data without restriction. Second, the system allows unrestricted data insertion but does not provide proper mechanisms for updating or removing data, which can lead to data inconsistency.

Third, the system architecture and code structure are not well organized, making it difficult to maintain, scale, or extend the system in the future.

Another significant limitation is that users who are not eligible (i.e., those who did not get a chance for admission) can still register in the system. This reduces the accuracy and trustworthiness of the admission process.

Finally, key features such as admission confirmation, automated message sending, and document verification are either missing or not fully implemented, which prevents the system from functioning as a complete automated admission solution.

1.3 Objectives

The primary objective of this project is to develop a user-friendly and reliable web-based admission system that ensures maximum accuracy and efficiency. The system aims to provide the following facilities:

- **Efficiency:** Automate the admission process to reduce manual effort, save time, and streamline data handling.
- **Accuracy:** Minimize errors by reducing manual data entry and ensuring proper handling of application data and documents.
- **Consistency:** Ensure that all data and applications are processed uniformly, reducing the risk of inconsistency and data loss.
- **Validation and Reliability:** Implement proper validation mechanisms to ensure that only eligible candidates can register and submit accurate information.
- **Improved Performance:** Enhance system performance and reliability by using modern technologies and optimized system architecture.
- **Improved Security:** Strengthen system security by incorporating updated security measures to protect user data and prevent unauthorized access.
- **Cost Effectiveness:** Reduce operational costs by minimizing the need for manual labor and paper-based processes.
- **Objectivity:** Ensure a fair and unbiased admission process by evaluating applicants based on predefined and objective criteria.

Chapter 2

Software Requirement Specification

2.1 Introduction

2.1.1 Purpose

The primary purpose of the JUST Undergraduate Final Admission System (JUST-UFAS) is to provide a secure, efficient, and user-friendly platform for completing the final admission process. The previous manual and semi-automated system had several limitations, including lack of proper validation, poor code quality, absence of structured design patterns, and no mechanism to enforce admission deadlines.

This system is the first structured and fully integrated solution developed by the JUST Undergraduate Admission Committee. It aims to eliminate existing issues by introducing proper validation, deadline control, role-based access, and a more maintainable and scalable architecture.

This Software Requirement Specification (SRS) document describes the complete workflow of the final admission process, including:

- Applicant interaction through online application form submission.
- Verification and clearance by the Dean Office with required documents.
- Bank receipt verification by the Hall Office.
- Medical examination and reporting by the Medical Center.
- Final approval and document collection by the Registrar Office.

- Administrative control for managing users, roles, and system operations.

2.1.2 Document Conventions

The following conventions are used throughout this SRS document:

- Section headings are numbered hierarchically.
- **Bold text** indicates important terms or concepts.
- All figures and tables are labeled with appropriate names and numbering.
- The following abbreviations are used:
 - JUST: Jashore University of Science and Technology.
 - SRS: Software Requirements Specification.
 - JUST-UFAS: JUST Undergraduate Final Admission System.

2.1.3 Intended Audience and Reading Suggestions

This document follows IEEE SRS standards and is intended for:

- Admission Committee members
- Project Supervisor
- Chairman of the CSE Department
- Internal and External Viva Board Members
- Future Developers and System Maintainers
- Any stakeholders interested in understanding the system

Readers are encouraged to begin with the Introduction and Overall Description sections before proceeding to detailed requirements.

2.2 Overall Description

2.2.1 Product Perspective

The JUST-UFAS is a web-based automated admission management system designed to replace the existing manual and partially digital processes. The previous system lacked critical features such as proper validation mechanisms, deadline enforcement, and structured system architecture.

The proposed system introduces:

- Strong input validation to prevent invalid or incomplete submissions.
- Deadline control to restrict registration and updates after the admission deadline.
- Role-based access control to ensure secure and authorized operations.
- A modular and maintainable architecture following modern design patterns.
- Real-time tracking of application progress across different offices.
- Scalability to handle a large number of concurrent applicants.

2.2.2 Product Functions

The major functionalities of the JUST-UFAS are as follows:

- **Admin Module:**
 - Manage users and roles.
 - Control system operations (open/close admission).
 - Monitor overall admission progress.
- **Dean Office Module:**
 - Verify documents and validate applicants.
 - Approve or reject applications.
 - Export applicant data.
- **Hall Office Module:**
 - Verify bank receipts.

- Manage residential clearance.
- **Medical Center Module:**
 - Record medical data.
 - Generate reports and provide clearance.
- **Registrar Office Module:**
 - Final admission approval.
 - Document collection and archiving.
- **Applicant Module:**
 - Registration and profile management.
 - Application form submission.
 - Document upload and download.

2.3 User Classes and Characteristics

The system includes multiple user roles, each with specific responsibilities. All users are expected to have basic knowledge of computers and web browsers.

Admin:

- Full system control.
- Manage users and roles.
- Control admission deadlines.

Dean Office:

- Verify documents.
- Approve or reject applications.

Hall Office:

- Verify payment and clearance.

Medical Office:

- Generate medical reports.

Registrar Office:

- Final admission approval.

Applicant:

- Submit application.
- Upload documents.
- Track status.

2.4 Constraints

The system operates under the following constraints:

- Admission is strictly controlled by deadlines.
- No registration or updates allowed after deadline.
- All inputs must pass validation rules.
- Role-based access control is enforced.
- Requires stable internet connection.
- Security measures must protect user data.

2.5 External Interface Requirements

2.5.1 User Interfaces

The system provides a web-based user interface designed for simplicity and usability. Interfaces are role-based, ensuring that each user (Admin, Dean Office, Hall Office, Medical Office, Registrar Office, Applicant) can only access permitted features.

The interface includes:

- Clear navigation and dashboard
- Form validation with proper instructions
- Role-specific panels for efficient task completion

2.5.2 Integration with Existing Data

The system supports integration with pre-existing admission data provided by the administration in Excel format. The admin uploads this data, which is then used to generate applicant accounts.

2.5.3 Integration with Internal Modules

All modules (Dean, Hall, Medical, Registrar) are interconnected and share a centralized database to ensure real-time updates and consistency.

2.5.4 Authentication and Authorization

The system implements secure authentication and role-based authorization:

- Admin provides login credentials to users
- Applicants log in using GST Application ID
- Access control ensures users can only perform authorized actions

2.6 System Features: Admin Management

2.6.1 Description and Priority

The Admin module has the highest priority and controls the entire system. The admin uploads applicant data (Excel file), manages users, controls admission deadlines, and monitors all activities.

2.6.2 Functional Requirements

Authentication

- Input: Username and password
- Output: Secure login access

Upload Applicant Data

- Input: Excel file containing applicant information
- Output: System generates applicant accounts

Manage Users

- View, update, and delete users

Update Applicant Information

- Modify applicant data if required

Control Admission

- Open or close admission system

View Reports

- Monitor admission progress and seat status

2.7 System Features: Dean Office

2.7.1 Description

The Dean Office verifies applicant information and approves or rejects applications based on eligibility criteria.

2.7.2 Functional Requirements

- Search applicant using ID
- View applicant profile
- Verify documents
- Approve or reject applicant
- View applicant lists (all, pending, approved)

2.8 System Features: Hall Office

2.8.1 Description

The Hall Office verifies payment and residential clearance.

2.8.2 Functional Requirements

- Search applicant
- View profile
- Verify bank receipt
- Approve applicant

2.9 System Features: Medical Office

2.9.1 Description

The Medical Office handles physical examination and medical clearance.

2.9.2 Functional Requirements

- Search applicant
- Insert medical data
- Generate report
- Approve applicant

2.10 System Features: Registrar Office

2.10.1 Description

The Registrar Office gives final admission approval and stores documents.

2.10.2 Functional Requirements

- Search applicant
- Receive documents
- Final approval
- Generate reports

2.11 System Features: Applicant

2.11.1 Description

Applicants do not register manually. Their accounts are created by admin-uploaded data. Applicants log in, complete their profile, and download the application form.

2.11.2 Functional Requirements

- Login using provided credentials
- Complete application form
- Upload required documents
- Download application form

2.12 Non-Functional Requirements

2.12.1 Performance

The system should handle a large number of users efficiently.

2.12.2 Security

Sensitive data must be protected using authentication and authorization.

2.12.3 Scalability

The system should support increasing users without performance loss.

2.12.4 Maintainability

The system should be easy to update and maintain.

2.12.5 Usability

The system must be user-friendly and easy to operate.

2.12.6 Reliability

The system should ensure high availability and minimal downtime.

Chapter 3

System Design and Methodology

3.1 Use Case Diagram

3.1.1 User Management

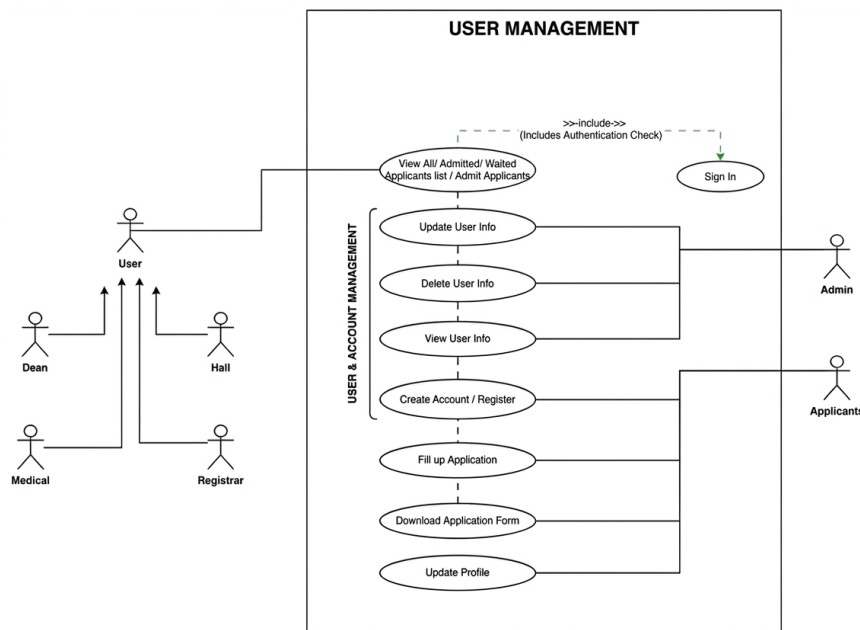


Figure 3.1: Use Case Diagram of User Management

The User Management use case diagram consists of six primary actors: Admin, Dean Office, Hall Office, Medical Office, Registrar Office, and Applicant.

3.1.1.1 Sign in

This use case describes the authentication process for the *JUST Undergraduate Final Admission System*. It ensures that only authorized personnel and registered applicants can access the system's restricted functionalities.

Characteristics	Description
Use Case ID	UC-1.1
Use Case Name	Sign in
Primary Actors	Admin, Dean Office, Hall Office, Medical Office, Registrar Office, Applicant
Goal in Context	To gain authorized access to the system dashboard.
Precondition	The user must have a valid account with a Username and Password.
Post-condition	The user is redirected to their respective dashboard based on their role.
Triggers	User attempts to access a protected route or clicks the Sign-in link.
Scenario	1. The user navigates to the System Login page. 2. The user enters their Username/Email and Password. 3. The user clicks the Login button. 4. The Next.js backend validates the credentials against the PostgreSQL database using Prisma. 5. Upon successful validation, a secure session is established. 6. The user is granted access to the system dashboard.
Exception	Invalid Credentials: If the credentials do not match, the system displays an "Access Denied" error message.
Frequency of Use	Extremely High (Daily).

Table 3.1: Use Case Table: Sign in (UC-1.1)

3.1.1.2 View Applicant List

This use case describes the procedure for administrative actors to access and review the records of candidates who have applied for admission. It allows various administrative offices (Dean, Hall, Medical, Registrar) to filter and monitor the status of

applicants throughout the admission lifecycle.

Characteristics	Description
Use Case ID	UC-1.2
Use Case Name	View Applicant List
Primary Actors	Admin, Dean Office, Hall Office, Medical Office, Registrar Office
Goal in Context	To filter and view different applicant categories (All, Admitted, or Waiting).
Precondition	The user must be securely authenticated and signed into the administrative dashboard.
Post-condition	The requested list of applicants is fetched and displayed on the UI.
Triggers	Need to verify candidate status or proceed with departmental clearance.
Scenario	1. The user visits the Dashboard after a successful login. 2. The user clicks on the 'Applicant Management' or 'View List' option from the sidebar. 3. The user chooses a specific list category (e.g., All Applicants, Admitted, or Waiting). 4. The Next.js backend fetches the data from the PostgreSQL database through Prisma. 5. The system displays the applicant records in a responsive data table.
Exception	Access Denied: If the user's role does not have permission to view certain lists, the system shows an unauthorized access message.
Frequency of Use	High (Used frequently for administrative tasks).

Table 3.2: Use Case Table: View Applicant List (UC-1.2)

3.1.1.3 Update User Information

This use case describes the administrative process of modifying existing user data within the system. It ensures that the Admin can maintain accurate records for all institutional roles in the *JUST Undergraduate Final Admission System*, allowing for corrections in personal details, roles, or account status.

Characteristics	Description
Use Case ID	UC-1.3
Use Case Name	Update User Information
Primary Actors	Admin
Goal in Context	To modify, correct, or update the profile information of any registered user.
Precondition	Admin must be securely authenticated and signed into the administrative dashboard.
Post-condition	The updated user information is successfully persisted in the PostgreSQL database.
Triggers	Requirement to update contact details, institutional roles, or personal information of a user.
Scenario	1. The Admin navigates to the User Management section of the dashboard. 2. The Admin searches for the specific user and clicks on the Edit icon/button. 3. The system populates a form with the user's current data from the database. 4. The Admin modifies the necessary fields (e.g., Name, Email, Role, or Status). 5. The Admin clicks the Update or Save Changes button. 6. The Next.js backend performs validation and updates the record via Prisma. 7. A success notification is displayed to the Admin.
Exception	Validation Error: If a duplicate email or invalid data format is entered, the system prevents the update and shows an error message.
Frequency of Use	High (Used for regular data maintenance).

Table 3.3: Use Case Table: Update User Information (UC-1.3)

3.1.1.4 Delete User Information

This use case describes the administrative process of permanently removing a user's record from the *JUST Undergraduate Final Admission System*. It allows the Admin to maintain system integrity by deleting redundant, unauthorized, or incorrect accounts from the database.

Characteristics	Description
Use Case ID	UC-1.4
Use Case Name	Delete User Information
Primary Actors	Admin
Goal in Context	To permanently remove a specific user record from the system database.
Precondition	The Admin must be signed in and authorized with administrative privileges.
Post-condition	The user record is successfully removed from the PostgreSQL database via Prisma.
Triggers	Admin identifies an account that needs to be removed for maintenance or security reasons.
Scenario	1. The Admin navigates to the User Management section in the dashboard. 2. The system fetches the list of all registered users from the database. 3. The Admin identifies and selects the specific user to be deleted. 4. The Admin clicks the Delete button associated with the user. 5. A confirmation modal appears to prevent accidental deletion. 6. Upon confirmation, the backend executes the delete query using Prisma. 7. The system updates the UI and displays a success notification.
Exception	Deletion Failed: If the user is linked to critical active records, the system prevents deletion and shows a "Constraint Error."
Frequency of Use	Low to Medium (As needed for system cleanup).

Table 3.4: Use Case Table: Delete User Information (UC-1.4)

3.1.1.5 View User Information

This use case allows the Admin to view detailed profiles and access levels of all users (Applicants and Staff) registered within the system. It is essential for verifying

user identity, checking account status, and managing system-wide permissions in the *JUST Undergraduate Final Admission System*.

Characteristics	Description
Use Case ID	UC-1.5
Use Case Name	View User Information
Primary Actors	Admin
Goal in Context	To display the full profile details, account status, and role of a specific user.
Precondition	Admin must be authenticated and signed into the administrative dashboard.
Post-condition	Detailed user information is retrieved from the database and displayed on the screen.
Triggers	Admin needs to verify user credentials or investigate a specific account's activity.
Scenario	1. The Admin navigates to the User Management or All Users section in the dashboard. 2. The Admin uses the search bar to find a specific user by their ID, Name, or Email. 3. The Admin clicks on the 'View Details' or 'Profile' icon of the selected user. 4. The Next.js API fetches the specific user's record from the PostgreSQL database via Prisma. 5. The system renders a detailed profile view including personal, contact, and role information.
Exception	User Not Found: If the requested ID does not exist, the system displays a "Record Not Found" error message.
Frequency of Use	High (Used frequently for administrative oversight).

Table 3.5: Use Case Table: View User Information (UC-1.5)

3.1.1.6 Fill-up Application Form

This use case outlines the process for a registered applicant to submit their academic and personal details through the *JUST Undergraduate Final Admission System*. It

is a multi-step procedure designed to collect necessary documentation and information required for the final admission validation.

Characteristics	Description
Use Case ID	UC-1.6
Use Case Name	Fill-up Application Form
Primary Actors	Applicant
Goal in Context	To successfully submit the complete admission application form.
Precondition	The applicant must be signed in and should not have a previously submitted pending application.
Post-condition	The application data is stored in the database, and the status is set to "Pending" for office review.
Triggers	Applicant clicks on the "Apply Now" or "Fill-up Form" button on their dashboard.
Scenario	1. The applicant navigates to the application form section. 2. The system presents a multi-step form (Personal Info, Academic History, Document Upload). 3. The applicant enters all required information and uploads necessary files (Photo, Signature, etc.). 4. The applicant reviews the information for any errors. 5. The applicant clicks the Submit button. 6. The Next.js frontend sends the data to the API, which persists it in PostgreSQL via Prisma. 7. A success confirmation message is displayed to the applicant.
Exception	Validation Error: If required fields are empty or files are in an unsupported format, the system prompts the user to correct the errors.
Frequency of Use	Once per applicant (High volume during admission periods).

Table 3.6: Use Case Table: Fill-up Application Form (UC-1.6)

3.1.1.7 Download Application Form

This use case provides the applicant with the ability to generate and download a formal PDF version of their submitted application. It serves as an official record for the applicant, which is essential for the physical verification process at *Jashore University of Science and Technology (JUST)*.

Characteristics	Description
Use Case ID	UC-1.7
Use Case Name	Download Application Form
Primary Actors	Applicant
Goal in Context	To obtain a digital or physical copy of the completed admission form for official documentation.
Precondition	The applicant must be securely signed in and must have successfully submitted their admission application.
Post-condition	A PDF document is dynamically generated and successfully downloaded to the applicant's local device.
Triggers	The applicant clicks the Download PDF or Print Application button on their personal dashboard.
Scenario	1. The applicant navigates to their personal dashboard. 2. The system verifies the application status from the PostgreSQL database. 3. The applicant clicks the Download PDF button. 4. The Next.js server-side logic triggers the PDF generation engine using the applicant's stored data. 5. The system initiates the file download stream in the user's browser. 6. The applicant saves the generated PDF file for future reference and physical submission.
Exception	Generation Error: If there is a server-side failure or the database is unreachable, a "PDF Generation Failed" error is displayed.
Frequency of Use	High (Once per applicant after the successful submission of the form).

Table 3.7: Use Case Table: Download Application Form (UC-1.7)

3.1.2 Dean Office Management

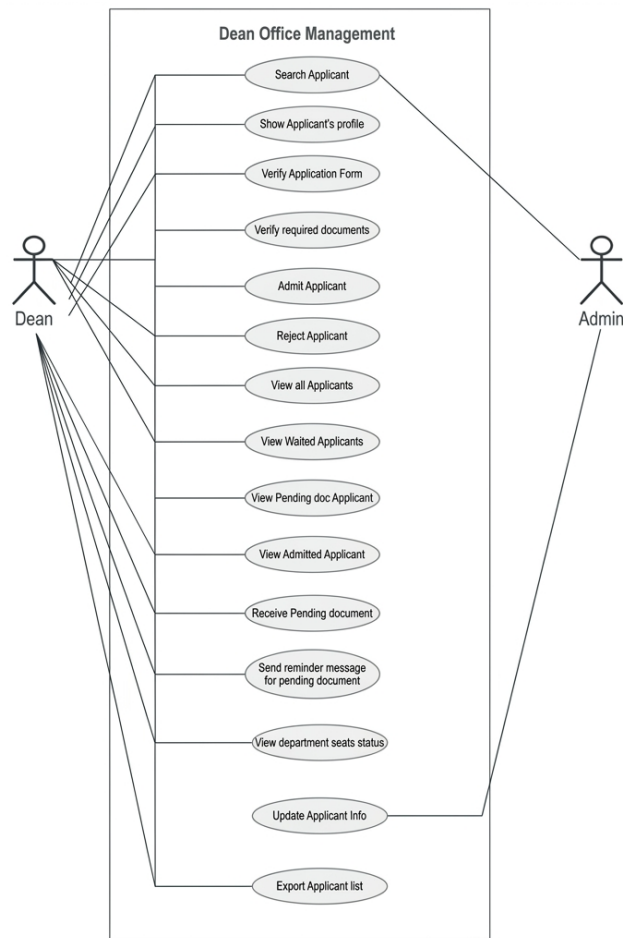


Figure 3.2: Use Case Diagram of Dean Office Management

The Dean Office Management use case diagram consists of two primary actors: Admin and Dean Office. This module controls various types of use cases including searching applicants, viewing applicant profiles, verifying application forms, verifying required documents, admitting applicants, rejecting applicants, and managing applicant lists.

It also provides functionalities to view all applicants, waited applicants, and admitted applicants with or without pending documents. Additionally, the system allows updating applicant information, handling pending documents, sending reminders, and monitoring department seat status.

3.1.2.1 Search Applicant

This use case describes how the Dean Office searches for an applicant.

Characteristics	Description
Use Case ID	UC-2.1
Use Case Name	Search Applicant
Primary Actors	Dean Office
Goal in Context	To search an applicant using ID or roll.
Precondition	Must be signed in and have Applicant ID or Roll.
Triggers	Need to search a specific applicant.
Scenario	1. Visit the Dean Office dashboard. 2. Enter Applicant ID or Roll in search box. 3. Press Enter.
Exception	Invalid role.
Frequency of Use	High

Table 3.8: Use Case Table: Search Applicant (UC-2.1)

3.1.2.2 Show Applicant Profile

This use case describes how the Dean Office views an individual applicant's profile. It provides a detailed view of the candidate's personal information and academic records, which is essential for making informed admission decisions.

Characteristics	Description
Use Case ID	UC-2.2
Use Case Name	Show Applicant Profile
Primary Actors	Dean Office
Goal in Context	To display the detailed profile of a specific applicant for review.
Precondition	The user must be signed in with Dean Office credentials and have searched for an applicant.
Post-condition	The system successfully displays the applicant's detailed data from the PostgreSQL database via Prisma ORM.
Triggers	Need to review specific academic or personal details of a candidate for admission processing.
Scenario	1. The user searches for a specific applicant from the applicant list. 2. The user clicks on the Profile or View Details button. 3. The system fetches the applicant data using Prisma. 4. The system displays the comprehensive profile including personal details, academic results, and payment records.
Exception	Data Not Found: If the record does not exist, an error message is shown. Access Denied: Unauthorized users are restricted.
Frequency of Use	High (During the admission screening phase).

Table 3.9: Use Case Table: Show Applicant Profile (UC-2.2)

3.1.2.3 Verify Application Form

This use case describes the procedure for the Dean Office to verify the submitted application form. It ensures that the applicant's data and uploaded documents are consistent with the university's requirements before granting final approval.

Characteristics	Description
Use Case ID	UC-2.3
Use Case Name	Verify Application Form
Primary Actors	Dean Office
Goal in Context	To successfully validate and approve the applicant's submitted application form.
Precondition	The user must be signed in with Dean Office privileges and an application must be pending review.
Post-condition	The application status is updated to 'Verified' in the PostgreSQL database and reflected on the system.
Triggers	Requirement to process and authorize candidate applications for the final admission list.
Scenario	1. The user searches for a specific applicant record from the pending list. 2. The user navigates to the Verification or Review section. 3. The system displays the submitted data and documents via the interface. 4. The user reviews the information and clicks the Approve or Verify button. 5. The system updates the status using Prisma ORM.
Exception	Data Mismatch: If the information is incorrect, the user may reject the form or request a correction from the applicant.
Frequency of Use	High (During peak admission cycles).

Table 3.10: Use Case Table: Verify Application Form (UC-2.3)

3.1.2.4 Verify Required Documents

This use case describes how the Dean Office verifies the physical or uploaded documents of an applicant. It ensures all academic certificates and required papers are authentic and meet the university's criteria.

Characteristics	Description
Use Case ID	UC-2.4
Use Case Name	Verify Required Documents
Primary Actors	Dean Office
Goal in Context	To successfully verify and confirm the validity of applicant documents.
Precondition	The user must be signed in and the applicant's record must be accessed.
Post-condition	The document status is updated to 'Verified' in the PostgreSQL database.
Triggers	Initiation of document screening during the admission process.
Scenario	1. Search for a specific applicant from the dashboard. 2. Click on the Verify Required Documents section. 3. Select the specific documents provided by the candidate. 4. Click the Verify button to confirm via Prisma ORM.
Exception	Invalid Role: Unauthorized staff attempt verification. Missing Docs: System flags missing files.
Frequency of Use	High

Table 3.11: Use Case Table: Verify Required Documents (UC-2.4)

3.1.2.5 Admit Applicant (Dean Clearance)

This use case allows the Dean Office to grant final clearance for admission after successful verification of all records.

Characteristics	Description
Use Case ID	UC-2.5
Use Case Name	Admit Applicant
Primary Actors	Dean Office
Goal in Context	To grant final admission clearance to the applicant.
Precondition	All documents must be verified and the applicant must be eligible.
Post-condition	Applicant status is changed to 'Admitted' and department seat count is updated.
Scenario	1. Search for an eligible applicant record. 2. Review the final summary of the candidate. 3. Click the Admit button. 4. System updates the record in PostgreSQL.
Exception	Seat capacity full or role unauthorized.
Frequency of Use	High

Table 3.12: Use Case Table: Admit Applicant (UC-2.5)

3.1.2.6 Reject Applicant

This use case describes the procedure for rejecting an applicant if significant discrepancies are found in their documents or eligibility.

Characteristics	Description
Use Case ID	UC-2.6
Use Case Name	Reject Applicant
Primary Actors	Dean Office
Goal in Context	To disqualify an applicant due to errors or non-compliance.
Scenario	1. Search for the specific applicant. 2. Click the Reject button. 3. Provide a reason for rejection (optional/required). 4. System updates the status to 'Rejected'.
Exception	Invalid role or accidental rejection of verified candidates.
Frequency of Use	High

Table 3.13: Use Case Table: Reject Applicant (UC-2.6)

3.1.2.7 View Applicant Lists (All, Waited, Admitted)

These use cases describe how the Dean Office monitors different categories of applicants through the dashboard sidebar.

Characteristics	Description
Use Case ID	UC-2.7, 2.8, 2.9
Use Case Name	View All / Waited / Admitted Applicants
Primary Actors	Dean Office
Scenario	1. Log in to the Dean Dashboard. 2. Select the respective item (All / Waited / Admitted) from the sidebar. 3. The system queries the PostgreSQL database via Prisma and displays the filtered list.
Frequency of Use	High

Table 3.14: Use Case Table: Applicant List Management (UC-2.7 - 2.9)

3.1.2.8 Pending Documents and Reminders

This section covers receiving pending documents and sending reminder messages to applicants who have yet to complete their documentation.

Characteristics	Description
Use Case ID	UC-2.11, 2.12, 2.13
Use Case Name	Manage Pending Documents & Send Reminders
Primary Actors	Dean Office
Scenario (Reminder)	1. Identify applicant with missing documents. 2. Select the missing items and click Send Message. 3. System triggers a notification to the applicant's contact.
Scenario (Receive)	1. Click Pending Documents Action from the sidebar. 2. Update the record upon receiving physical copies.

Table 3.15: Use Case Table: Pending Documents Management (UC-2.11 - 2.13)

3.1.2.9 Seats Status and Data Export

Characteristics	Description
Use Case ID	UC-2.14, 2.15
Use Case Name	View Seats Status & Export Lists
Primary Actors	Dean
Goal in Context	To monitor enrollment capacity and generate reports in CSV/Excel format.
Scenario (Export)	1. Navigate to the desired applicant list. 2. Click the Export button. 3. The system generates and downloads the file.

Table 3.16: Use Case Table: Monitoring and Export (UC-2.14 - 2.15)